Project 2: Distance Vector Routing Implementation

For this project, I implemented Distance Vector Routing using Bellman-Ford. The goal was to have each node find the shortest path to other nodes and use that information to send packets through multiple hops. I reused the Neighbor Discovery and Flooding modules from Project 1, but instead of using flooding to send regular packets, I added a Distance Vector module and used IP to forward packets along the selected route.

For Distance Vector, each node keeps a routing table that stores the destination, next hop, cost, and whether the route is valid. Every active neighbor has a cost of 1, and the node has a cost of 0 to itself. I used an INFINITY value of 16, so a route with a cost of 16 is considered unreachable. Nodes send their distance estimates to their active neighbors periodically, about every 10 seconds. When a node receives an update, it adds 1 to the advertised cost and checks whether that route is better than the one it already has. If the update comes from its current next hop, it also accepts a worse cost or an unreachable route. For routes with the same cost, I try to keep the current next hop.

I also implemented split horizon with poison reverse to help prevent routing loops. If a node reaches a destination through a particular neighbor, it advertises a cost of 16 for that destination when sending its routing information back to that neighbor. Other neighbors receive the actual cost. I send a separate advertisement to each active neighbor because the advertised costs can be different. Each packet can hold up to 6 destination and cost entries, so when the routing table is larger, the module sends the information in multiple packets instead of leaving entries out.

To handle changes in the network, the Distance Vector module continues checking which neighbors are active through Neighbor Discovery. It keeps track of when routing information was last received and expires learned information after 5 update intervals without a refresh. When a route becomes unreachable, it is kept for another 5 update intervals so the node can advertise its withdrawal before removing it. I also used periodic updates and short randomized triggered updates to help nodes learn changes without sending everything at once.

An important design decision was to keep Distance Vector and IP forwarding in separate modules. Distance Vector is responsible for maintaining the routing table, while IP looks up the next hop whenever it sends or forwards a packet. IP keeps the original source and destination and decreases the TTL when forwarding, so packets cannot keep circulating forever. I used a routing capacity of 32 destinations and 8 cached neighbors. In the simulations, I tested multi-hop pings, poison reverse, alternate paths, network partitions, larger routing tables, and packet loss. The routes changed when links failed and were learned again when communication was restored.